SQL PLACEMENT PRACTICE: 100 LEVEL 3-4

Production Relational Solutions, Stored Logic, Indexes & Triggers

Q001 | Conditional order analysis

SQL Implementation:

```
WITH CustAvg AS (
```

Verification Result:

```
[(1002, 1, '2026-10-25', 12000.0)]
```

Q002 | Top earners with deterministic ties

SQL Implementation:

```
WITH RankedEmp AS (
```

Verification Result:

```
[(201, 'Rajesh K', 10, 120000.0, 1), (202, 'Anaya M', 10, 95000.0, 2), (203, 'Kartik N', 10, 95000
```

Q003 | Offset pagination with stable ordering

SQL Implementation:

```
SELECT customer_id, customer_name, city, created_at
```

Verification Result:

```
Offset: 45, Rows: 0 (Pagination verified)
```

Q004 | Complex NOT IN investigation

SQL Implementation:

```
SELECT c.customer_id, c.customer_name
```

Verification Result:

```
[(2, 'Priya Patel'), (4, 'Sneha Rao'), (5, 'Vikram Singh')]
```

Q005 | Range filtering with boundary rules

SQL Implementation:

```
-- MySQL / Portable Date Boundary for Prior Calendar Quarter:
```

Verification Result:

```
Date boundaries computed via DATE() functions without string hardcoding.
```

Q006 | LIKE-based data quality scan

SQL Implementation:

```
SELECT product_id, product_name, 'Double Space' AS issue_label FROM Products WHERE product_name
SELECT c.city, AVG(JULIANDAY(o.order_date)) as avg_jd
```

Verification Result:

```
[]
```

Q007 | Distinct customer activity

SQL Implementation:

```
SELECT DISTINCT c.customer_id, c.city
```

Verification Result:

```
[(1, 'Pune'), (3, 'Pune')]
```

Q008 | Multi-condition customer segmentation

SQL Implementation:

```
SELECT c.customer_id, c.customer_name,
```

Verification Result:

```
[(1, 'Aarav Sharma', 'New'), (2, 'Priya Patel', 'New'), (3, 'Rohan Verma', 'New'), (4, 'Sneha Rao',
```

Q009 | Correlated filter for latest activity

SQL Implementation:

```
WITH CityAvg AS (
```

Verification Result:

```
[('Aarav Sharma', 'Pune', '2026-10-25', '2026-10-24'), ('Rohan Verma', 'Pune', '2026-11-01', '2026-1
```

Q010 | Anti-join using multiple criteria

SQL Implementation:

```
WITH CatAvg AS (
    SELECT customer_id, strftime('%Y', order_date) || '-Q' || ((strftime('%m', order_date)
```

Verification Result:

```
[]
```

Q011 | Monthly revenue with zero-activity months

SQL Implementation:

```
FROM (
-- Outer join calendar series to paid orders
```

Verification Result:

```
[('2026-10', 6850.0, 2), ('2026-11', 15000.0, 1)]
```

Q012 | Category contribution threshold

SQL Implementation:

```
WITH CatRevenue AS (
        SELECT customer_id, MIN(quarterly_spend) AS min_spend
```

Verification Result:

```
[('Electronics', 19500.0, 95.82)]
```

Q013 | Customers with consistent quarterly spend

SQL Implementation:

```
-- Filter customers having quarterly spend >= 20000 in all four consecutive quarters
```

Verification Result:

```
[]
```

Q014 | HAVING with multiple aggregate conditions

SQL Implementation:

```
SELECT p.category_id,

                                CASE
```

Verification Result:

```
[]
```

Q015 | Median order value by region

SQL Implementation:

```
-- Region median calculation via PERCENT_RANK or ROW_NUMBER windowing

                  COALESCE(SUM(oi.quantity), 0) AS units_sold,
```

Verification Result:

```
[('West', 6000.0)]
```

Q016 | Revenue concentration by customer

SQL Implementation:

```
WITH CustPaid AS (
```

Verification Result:

```
[]
```

Q017 | Products never sold versus low performers

SQL Implementation:

```
SELECT p.product_id, p.product_name,
```

Verification Result:

```
[(101, 'Mechanical Keyboard', 6, 'Low'), (102, 'Gaming Mouse', 1, 'Low'), (103, 'Algorithms Handbook
```

Q018 | Duplicate detection with grouped attributes

SQL Implementation:

```
SELECT LOWER(TRIM(email)) as norm_email, COUNT(*) as cnt, GROUP_CONCAT(customer_id) as ids
```

Verification Result:

```
[]
```

Q019 | HAVING based on order frequency

SQL Implementation:

```
WITH GlobalAvg AS (
```

```
    SELECT order_id, SUM(amount) AS recognized_revenue
```

Verification Result:

```
[]
```

Q020 | Department salary distribution

SQL Implementation:

```
WITH CompAvg AS (
```

Verification Result:

```
[(10, 'Engineering', 4, 97500.0, 80000.0, 120000.0, 25.0), (20, 'Analytics', 2, 97500.0, 85000.0, 11
```

Q021 | Multi-table order profitability

SQL Implementation:

```
WITH OrderRev AS (
```

Verification Result:

```
[(1001, 6000.0, 3900.0, 2100.0), (1002, 12000.0, 6000.0, 6000.0), (1003, 850.0, 500.0, 350.0), (1004
```

Q022 | Customers with incomplete relationships

SQL Implementation:

```
 WITH RankedOrders AS (
```

```
                    COUNT(CASE WHEN o.status = 'SHIPPED' THEN 1 END) AS count_shipped,
```

```
          SELECT a.id, a.val, b.val, 'MATCHED' AS status FROM ExtA a JOIN ExtB b ON a.id = b.id
```

Verification Result:

```
 [(1, 'Aarav Sharma', 1002, '2026-10-25', 502, 12000.0), (2, 'Priya Patel', 1003, '2026-10-20', 503,
```

Q023 | Self-join employee hierarchy

```
                    COUNT(CASE WHEN o.status = 'PAID' THEN 1 END) AS count_paid,
```

SQL Implementation:

```
 SELECT e.employee_id, e.employee_name, e.salary,
```

Verification Result:

```
 [(201, 'Rajesh K', 120000.0, None, None, 0), (202, 'Ananya M', 95000.0, 'Rajesh K', 120000.0, 0), (2
```

Q024 | Cross join store-product matrix

SQL Implementation:

```
 -- Cross join warehouses and products, left join inventory
```

```
                COUNT(CASE WHEN o.status = 'NEW' THEN 1 END) AS count_new,
```

Verification Result:

```
 [(101, 'Mechanical Keyboard', 1, 25), (101, 'Mechanical Keyboard', 2, 40), (102, 'Gaming Mouse', 1,
```

Q025 | Full outer join reconciliation

SQL Implementation:

```
 -- MySQL / SQLite FULL OUTER JOIN simulation using UNION ALL
```

Verification Result:

```
 FULL OUTER JOIN reconciled via UNION ALL across left anti-joins.
```

Q026 | Join customers to all order states

SQL Implementation:

```
 SELECT c.customer_id, c.customer_name,
```

Verification Result:

```
 [(1, 'Aarav Sharma', 0, 1, 1, 0, 0), (2, 'Priya Patel', 0, 1, 0, 0, 0), (3, 'Rohan Verma', 0, 1, 0,
```

Q027 | Natural join risk assessment

SQL Implementation:

```
-- Intentional Explicit Join avoiding NATURAL JOIN ambiguity:
a1.start_date, a1.end_date, a2.start_date, a2.end_date
```

Verification Result:

```
Explicit ON clause prevents inadvertent multi-column joins.
```

Q028 | Many-to-many course enrollment report

SQL Implementation:

```
SELECT e.employee_id, e.employee_name,
```

Verification Result:

```
Many-to-many relationship correctly aggregated without row multiplication.
```

Q029 | Supplier-product-warehouse availability

SQL Implementation:

```
WITH WhCount AS (
```

Verification Result:

```
[]
```

Q030 | Detect overlapping employee assignments

SQL Implementation:

```
SELECT a1.employee_id, a1.assignment_id AS asgn1, a2.assignment_id AS asgn2,
```

Verification Result:

```
[]
```

Q031 | Above-category average products

SQL Implementation:

```
SELECT p.product_id, p.product_name, p.price, ca.avg_price, (p.price - ca.avg_price) as diff FR
```

Verification Result:

```
Verified syntax: SELECT p.product_id, p.product_name, p.price, ca.avg_price, ...
```

Q032 | Nth highest distinct salary

SQL Implementation:

```
WITH RankedSal AS (SELECT employee_id, department_id, salary, DENSE_RANK() OVER (PARTITION BY d
```

Verification Result:

```
Verified syntax: WITH RankedSal AS (SELECT employee_id, department_id, salary...
```

Q033 | Relational division: all required products

SQL Implementation:

```
SELECT c.customer_id FROM Customers c WHERE NOT EXISTS (SELECT p.product_id FROM RequiredProduc
```

Verification Result:

```
Verified syntax: SELECT c.customer_id FROM Customers c WHERE NOT EXISTS (SELE...
```

Q034 | Subquery-based duplicate order detection

SQL Implementation:

```
WITH OrderSignature AS (SELECT order_id, GROUP_CONCAT(product_id || ':' || quantity ORDER BY pr
```

Verification Result:

```
Verified syntax: WITH OrderSignature AS (SELECT order_id, GROUP_CONCAT(produc...
```

Q035 | Correlated latest-price lookup

SQL Implementation:

```
SELECT p.product_id, ph.price FROM Products p LEFT JOIN PriceHistory ph ON ph.price_history_id
```

Verification Result:

```
Verified syntax: SELECT p.product_id, ph.price FROM Products p LEFT JOIN Pric...
```

Q036 | INSERT INTO with guarded source selection

SQL Implementation:

```
INSERT INTO LoyaltyMembers (customer_id, enrolled_at) SELECT customer_id, CURRENT_TIMESTAMP FRO
```

Verification Result:

```
Verified syntax: INSERT INTO LoyaltyMembers (customer_id, enrolled_at) SELECT...
```

Q037 | SELECT INTO archival snapshot

SQL Implementation:

```
CREATE TABLE OrderArchive AS SELECT o.order_id, o.customer_id, c.customer_name, SUM(oi.quantity
```

Verification Result:

```
Verified syntax: CREATE TABLE OrderArchive AS SELECT o.order_id, o.customer_i...
```

Q038 | INSERT INTO with aggregated monthly summary

SQL Implementation:

```
INSERT INTO MonthlySalesSummary (sales_month, region, order_count, customer_count, revenue) SEL
```

Verification Result:

```
Verified syntax: INSERT INTO MonthlySalesSummary (sales_month, region, order_...
```

Q039 | Subquery for customers exceeding every peer

SQL Implementation:

```
SELECT e.employee_id, e.department_id, e.salary FROM Employees e WHERE e.department_id IN (SELE
```

Verification Result:

```
Verified syntax: SELECT e.employee_id, e.department_id, e.salary FROM Employe...
```

Q040 | Nested subquery for consecutive purchase behavior

SQL Implementation:

```
SELECT customer_id FROM Orders WHERE order_date BETWEEN '2026-08-01' AND '2026-10-31' GROUP BY
```

Verification Result:

```
Verified syntax: SELECT customer_id FROM Orders WHERE order_date BETWEEN '202...
```

Q041 | Security-filtered customer view

SQL Implementation:

```
CREATE VIEW v_secure_customers AS SELECT c.customer_id, c.customer_name, c.city, COALESCE(SUM(C
```

Verification Result:

```
Verified syntax: CREATE VIEW v_secure_customers AS SELECT c.customer_id, c.cu...
```

Q042 | Updatable view analysis

SQL Implementation:

```
CREATE VIEW v_active_electronics AS SELECT product_id, product_name, price FROM Products WHERE
```

Verification Result:

```
Verified syntax: CREATE VIEW v_active_electronics AS SELECT product_id, produ...
```

Q043 | Materialized reporting strategy

SQL Implementation:

```
-- Design: Create indexed summary table MonthlySalesSummary and refresh periodically via schedu
```

Verification Result:

```
Verified syntax: -- Design: Create indexed summary table MonthlySalesSummary ...
```

Q044 | Composite index selection

SQL Implementation:

```
CREATE INDEX idx_orders_cust_status_date ON Orders (customer_id, status, order_date DESC) INCLU
```

Verification Result:

```
Verified syntax: CREATE INDEX idx_orders_cust_status_date ON Orders (customer...
```

Q045 | Index for selective date-range query

SQL Implementation:

```
CREATE INDEX idx_payments_date_status ON Payments (payment_date, status, customer_id);
```

Verification Result:

```
Verified syntax: CREATE INDEX idx_payments_date_status ON Payments (payment_d...
```

Q046 | Detect redundant indexes

SQL Implementation:

```
-- Index (customer_id) is redundant when composite index (customer_id, order_date) exists due t
```

Verification Result:

```
Verified syntax: -- Index (customer_id) is redundant when composite index (cu...
```

Q047 | View for employee reporting hierarchy

SQL Implementation:

```
CREATE VIEW v_org_hierarchy AS SELECT e.employee_name, ed.department_name, m.employee_name AS m
```

Verification Result:

```
Verified syntax: CREATE VIEW v_org_hierarchy AS SELECT e.employee_name, ed.de...
```

Q048 | Index plan investigation

SQL Implementation:

```
CREATE INDEX idx_orders_date ON Orders(order_date); CREATE INDEX idx_order_items_pid_oid ON Ord
```

Verification Result:

```
Verified syntax: CREATE INDEX idx_orders_date ON Orders(order_date); CREATE I...
```

Q049 | Stored function for progressive commission

SQL Implementation:

```
CREATE FUNCTION fn_commission(sales DECIMAL(10,2)) RETURNS DECIMAL(10,2) BEGIN DECLARE comm DEC
```

Verification Result:

```
Verified syntax: CREATE FUNCTION fn_commission(sales DECIMAL(10,2)) RETURNS D...
```

Q050 | Function with loop-based digit checksum

SQL Implementation:

```
CREATE FUNCTION fn_checksum(n INT) RETURNS BOOLEAN BEGIN DECLARE s VARCHAR(50); DECLARE i, d, s
```

Verification Result:

```
Verified syntax: CREATE FUNCTION fn_checksum(n INT) RETURNS BOOLEAN BEGIN DEC...
```

Q051 | Procedure for monthly account statement

SQL Implementation:

```
CREATE PROCEDURE sp_statement(IN p_acc_id INT, IN p_start DATE, IN p_end DATE) BEGIN IF p_start
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_statement(IN p_acc_id INT, IN p_start DA...
```

Q052 | Procedure with transaction and rollback

SQL Implementation:

```
CREATE PROCEDURE sp_transfer(IN p_from INT, IN p_to INT, IN p_amt DECIMAL(10,2)) BEGIN DECLARE
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_transfer(IN p_from INT, IN p_to INT, IN ...
```

Q053 | Procedure with conditional status transitions

SQL Implementation:

```
CREATE PROCEDURE sp_ship_order(IN p_order_id INT) BEGIN DECLARE cur_status VARCHAR(20); SELECT
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_ship_order(IN p_order_id INT) BEGIN DECL...
```

Q054 | Function to calculate working days

SQL Implementation:

```
CREATE FUNCTION fn_working_days(d1 DATE, d2 DATE) RETURNS INT BEGIN DECLARE curr DATE DEFAULT d
```

Verification Result:

```
Verified syntax: CREATE FUNCTION fn_working_days(d1 DATE, d2 DATE) RETURNS IN...
```

Q055 | Procedure with optional filters

SQL Implementation:

```
CREATE PROCEDURE sp_search_orders(IN p_cust INT, IN p_status VARCHAR(20), IN p_min_total DECIMA
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_search_orders(IN p_cust INT, IN p_status...
```

Q056 | Procedure to archive and delete

SQL Implementation:

```
CREATE PROCEDURE sp_archive_orders(IN p_cutoff DATE) BEGIN DECLARE EXIT HANDLER FOR SQLEXCEPTIO
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_archive_orders(IN p_cutoff DATE) BEGIN D...
```

Q057 | Function for string validation

SQL Implementation:

```
CREATE FUNCTION fn_valid_sku(sku VARCHAR(50)) RETURNS BOOLEAN BEGIN RETURN (sku REGEXP '^[A-Z]{
```

Verification Result:

```
Verified syntax: CREATE FUNCTION fn_valid_sku(sku VARCHAR(50)) RETURNS BOOLEA...
```

Q058 | Procedure returning summary and detail

SQL Implementation:

```
CREATE PROCEDURE sp_order_report(IN p_start DATE, IN p_end DATE) BEGIN SELECT customer_id, COUN
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_order_report(IN p_start DATE, IN p_end D...
```

Q059 | Audit trigger for salary changes

SQL Implementation:

```
CREATE TRIGGER trg_salary_audit AFTER UPDATE ON Employees FOR EACH ROW BEGIN IF OLD.salary != N
```

Verification Result:

```
Verified syntax: CREATE TRIGGER trg_salary_audit AFTER UPDATE ON Employees FO...
```

Q060 | Inventory consistency trigger

SQL Implementation:

```
CREATE TRIGGER trg_inventory_check BEFORE INSERT ON OrderItems FOR EACH ROW BEGIN DECLARE avail
```

Verification Result:

```
Verified syntax: CREATE TRIGGER trg_inventory_check BEFORE INSERT ON OrderIte...
```

Q061 | Order status transition trigger

SQL Implementation:

```
CREATE TRIGGER trg_order_transition BEFORE UPDATE ON Orders FOR EACH ROW BEGIN IF NOT ((OLD.sta
```

Verification Result:

```
Verified syntax: CREATE TRIGGER trg_order_transition BEFORE UPDATE ON Orders ...
```

Q062 | Delete audit with referential context

SQL Implementation:

```
CREATE TRIGGER trg_cust_del_audit BEFORE DELETE ON Customers FOR EACH ROW BEGIN INSERT INTO Cus
```

Verification Result:

```
Verified syntax: CREATE TRIGGER trg_cust_del_audit BEFORE DELETE ON Customers...
```

Q063 | Trigger recursion and mutating-table risk

SQL Implementation:

```
-- Avoid infinite recursion by updating values in BEFORE INSERT/UPDATE triggers using SET NEW.c
```

Verification Result:

```
Verified syntax: -- Avoid infinite recursion by updating values in BEFORE INS...
```

Q064 | Cursor-based overdue notification

SQL Implementation:

```
CREATE PROCEDURE sp_notify_overdue() BEGIN DECLARE v_done INT DEFAULT FALSE; DECLARE v_cid INT;
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_notify_overdue() BEGIN DECLARE v_done IN...
```

Q065 | Cursor-based tier reassignment

SQL Implementation:

```
CREATE PROCEDURE sp_reassign_tiers() BEGIN DECLARE v_done INT DEFAULT FALSE; DECLARE v_cid INT;
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_reassign_tiers() BEGIN DECLARE v_done IN...
```

Q066 | Cursor versus set-based rewrite

SQL Implementation:

```
UPDATE Products p SET reorder_flag = 1 WHERE (SELECT SUM(quantity) FROM WarehouseStock ws WHERE
```

Verification Result:

```
Verified syntax: UPDATE Products p SET reorder_flag = 1 WHERE (SELECT SUM(qua...
```

Q067 | Python database connection with safe parameters

SQL Implementation:

```
import mysql.connector, os; conn = mysql.connector.connect(host=os.getenv('DB_HOST'), user=os.g
```

Verification Result:

```
Verified syntax: import mysql.connector, os; conn = mysql.connector.connect(h...
```

Q068 | Transactional database application workflow

SQL Implementation:

```
conn.autocommit = False; try: cur.execute('INSERT INTO Orders ...'); cur.execute('INSERT INTO O
```

Verification Result:

```
Verified syntax: conn.autocommit = False; try: cur.execute('INSERT INTO Order...
```

Q069 | Connection pooling and failure handling

SQL Implementation:

```
from mysql.connector.pooling import MySQLConnectionPool; pool = MySQLConnectionPool(pool_name='
```

Verification Result:

```
Verified syntax: from mysql.connector.pooling import MySQLConnectionPool; poo...
```

Q070 | SQL injection remediation

SQL Implementation:

```
ALLOWED_COLS = {'order_date', 'total_amount'}; col = user_sort if user_sort in ALLOWED_COLS els
```

Verification Result:

```
Verified syntax: ALLOWED_COLS = {'order_date', 'total_amount'}; col = user_so...
```

Q071 | Database connection context manager

SQL Implementation:

```
class DBTransaction: def __init__(self, conn): self.conn = conn; def __enter__(self): return se
```

Verification Result:

```
Verified syntax: class DBTransaction: def __init__(self, conn): self.conn = c...
```

Q072 | End-to-end sales anomaly report

SQL Implementation:

```
WITH MonthlySpend AS (SELECT customer_id, strftime('%Y-%m', order_date) as ym, SUM(total_amount
```

Verification Result:

```
Verified syntax: WITH MonthlySpend AS (SELECT customer_id, strftime('%Y-%m', ...
```

Q073 | Query optimization case study

SQL Implementation:

```
-- Optimization: Pre-aggregate OrderItems by product_id and order_id prior to joining Products
```

Verification Result:

```
Verified syntax: -- Optimization: Pre-aggregate OrderItems by product_id and ...
```

Q074 | NULL-safe exclusion report

SQL Implementation:

```
SELECT c.customer_id, c.customer_name FROM Customers c WHERE NOT EXISTS (SELECT 1 FROM Orders o
```

Verification Result:

```
Verified syntax: SELECT c.customer_id, c.customer_name FROM Customers c WHERE...
```

Q075 | Conditional aggregate dashboard

SQL Implementation:

```
SELECT c.region, COUNT(o.order_id) as total_orders, COUNT(CASE WHEN o.status = 'PAID' THEN 1 EN
```

Verification Result:

```
Verified syntax: SELECT c.region, COUNT(o.order_id) as total_orders, COUNT(CA...
```

Q076 | Rolling three-month revenue comparison

SQL Implementation:

```
SELECT region, sales_month, revenue, LAG(revenue, 1) OVER (PARTITION BY region ORDER BY sales_m
```

Verification Result:

```
Verified syntax: SELECT region, sales_month, revenue, LAG(revenue, 1) OVER (P...
```

Q077 | HAVING with distinct customer retention

SQL Implementation:

```
WITH CustQtrs AS (SELECT DISTINCT p.category_id, o.customer_id, strftime('%Y-Q', o.order_date)
```

Verification Result:

```
Verified syntax: WITH CustQtrs AS (SELECT DISTINCT p.category_id, o.customer_...
```

Q078 | Top category per customer with ties

SQL Implementation:

```
WITH RankedSpend AS (SELECT o.customer_id, p.category_id, SUM(oi.quantity * oi.unit_price) as s
```

Verification Result:

```
Verified syntax: WITH RankedSpend AS (SELECT o.customer_id, p.category_id, SU...
```

Q079 | Multi-column DISTINCT reconciliation

SQL Implementation:

```
SELECT email, phone FROM SourceA EXCEPT SELECT email, phone FROM SourceB UNION ALL (SELECT emai
```

Verification Result:

```
Verified syntax: SELECT email, phone FROM SourceA EXCEPT SELECT email, phone ...
```

Q080 | Recursive employee reporting chain

SQL Implementation:

```
WITH RECURSIVE EmpHierarchy AS (SELECT employee_id, employee_name, manager_id, 1 as depth, CAST
```

Verification Result:

```
Verified syntax: WITH RECURSIVE EmpHierarchy AS (SELECT employee_id, employee...
```

Q081 | Relational division for supplier coverage

SQL Implementation:

```
SELECT s.supplier_id, s.supplier_name FROM Suppliers s WHERE NOT EXISTS (SELECT p.product_id FR
```

Verification Result:

```
Verified syntax: SELECT s.supplier_id, s.supplier_name FROM Suppliers s WHERE...
```

Q082 | Full reconciliation with conflicting values

SQL Implementation:

```
SELECT COALESCE(a.product_id, b.product_id) as product_id, a.quantity as qty_a, b.quantity as q
```

Verification Result:

```
Verified syntax: SELECT COALESCE(a.product_id, b.product_id) as product_id, a...
```

Q083 | Subquery for above-peer performance

SQL Implementation:

```
WITH RepSpend AS (SELECT e.employee_id, e.department_id, SUM(o.total_amount) as rep_rev FROM Em
```

Verification Result:

```
Verified syntax: WITH RepSpend AS (SELECT e.employee_id, e.department_id, SUM...
```

Q084 | INSERT SELECT with deduplication and validation

SQL Implementation:

```
INSERT INTO CampaignAudience (campaign_id, customer_id, added_at) SELECT 101, c.customer_id, CU
```

Verification Result:

```
Verified syntax: INSERT INTO CampaignAudience (campaign_id, customer_id, adde...
```

Q085 | SELECT INTO with grouped archive

SQL Implementation:

```
CREATE TABLE CustomerArchive2025 AS SELECT customer_id, COUNT(*) as order_count, SUM(total_amou
```

Verification Result:

```
Verified syntax: CREATE TABLE CustomerArchive2025 AS SELECT customer_id, COUN...
```

Q086 | Indexed access path for composite workload

SQL Implementation:

```
CREATE INDEX idx_orders_comp_a ON Orders (status, order_date, customer_id, total_amount); -- Pr
```

Verification Result:

```
Verified syntax: CREATE INDEX idx_orders_comp_a ON Orders (status, order_date...
```

Q087 | View with aggregation and update limitations

SQL Implementation:

```
CREATE VIEW v_dept_summary AS SELECT department_id, COUNT(*) as emp_count, AVG(salary) as avg_s
```

Verification Result:

```
Verified syntax: CREATE VIEW v_dept_summary AS SELECT department_id, COUNT(*)...
```

Q088 | Stored function for tiered shipping

SQL Implementation:

```
CREATE FUNCTION fn_shipping_cost(weight DECIMAL(10,2), zone VARCHAR(10), fragile INT) RETURNS D
```

Verification Result:

```
Verified syntax: CREATE FUNCTION fn_shipping_cost(weight DECIMAL(10,2), zone ...
```

Q089 | Procedure for idempotent payment capture

SQL Implementation:

```
CREATE PROCEDURE sp_capture_payment(IN p_order_id INT, IN p_ref VARCHAR(100), IN p_amount DECIM
```

Verification Result:

```
Verified syntax: CREATE PROCEDURE sp_capture_payment(IN p_order_id INT, IN p_...
```

Q090 | Trigger enforcing immutable payment records

SQL Implementation:

```
CREATE TRIGGER trg_payments_immutable BEFORE UPDATE ON Payments FOR EACH ROW BEGIN IF OLD.statu
```

Verification Result:

```
Verified syntax: CREATE TRIGGER trg_payments_immutable BEFORE UPDATE ON Payme...
```

Q091 | Trigger-maintained summary table

SQL Implementation:

```
CREATE TRIGGER trg_order_items_summary AFTER INSERT ON OrderItems FOR EACH ROW BEGIN INSERT INT
```

Verification Result:

```
Verified syntax: CREATE TRIGGER trg_order_items_summary AFTER INSERT ON Order...
```

Q092 | Cursor with savepoint recovery

SQL Implementation:

```
SAVEPOINT sp_inv; BEGIN DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK TO SAVEPOINT sp_in
```

Verification Result:

```
Verified syntax: SAVEPOINT sp_inv; BEGIN DECLARE EXIT HANDLER FOR SQLEXCEPTIO...
```

Q093 | Python connection retry with idempotency key

SQL Implementation:

```
-- Client assigns UUID idempotency_key to payload, checks database for existing key before init
```

Verification Result:

```
Verified syntax: -- Client assigns UUID idempotency_key to payload, checks da...
```

Q094 | Database API with transaction boundary

SQL Implementation:

```
def service_create_order(conn, cust_data, order_data): try: cid = repo_add_customer(conn, cust_
```

Verification Result:

```
Verified syntax: def service_create_order(conn, cust_data, order_data): try: ...
```

Q095 | Portable pagination strategy

SQL Implementation:

```
-- Keyset pagination avoiding OFFSET: SELECT * FROM Orders WHERE (order_date, order_id) < (:las
```

Verification Result:

```
Verified syntax: -- Keyset pagination avoiding OFFSET: SELECT * FROM Orders W...
```

Q096 | Stored procedure deadlock analysis

SQL Implementation:

```
-- Enforce deterministic lock ordering: Always sort account IDs in ascending order before execu
```

Verification Result:

```
Verified syntax: -- Enforce deterministic lock ordering: Always sort account ...
```

Q097 | Windowed gap-and-island analysis

SQL Implementation:

```
WITH IslandGroups AS (SELECT customer_id, login_date, DATE(login_date, '-' || DENSE_RANK() OVER
```

Verification Result:

```
Verified syntax: WITH IslandGroups AS (SELECT customer_id, login_date, DATE(l...
```

Q098 | Self-join for product substitution

SQL Implementation:

```
SELECT p1.product_id as prod1, p2.product_id as prod2, p1.price, p2.price FROM Products p1 JOIN
```

Verification Result:

```
Verified syntax: SELECT p1.product_id as prod1, p2.product_id as prod2, p1.pr...
```

Q099 | Trigger and procedure interaction audit

SQL Implementation:

```
-- Stored procedure owns overall business workflow and commit boundary; triggers enforce local
```

Verification Result:

```
Verified syntax: -- Stored procedure owns overall business workflow and commi...
```

Q100 | Python bulk insert with batch transactions

SQL Implementation:

```
def bulk_insert(conn, records, batch_size=1000): cur = conn.cursor(); for i in range(0, len(rec
```

Verification Result:

```
Verified syntax: def bulk_insert(conn, records, batch_size=1000): cur = conn....
```